SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

(10 Marks)

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I Y. Thanushma / 192425355 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y. Thanushma

RUBRICS FOR CALCULATION

Name :- Y. Thanushma

Registration Number :- 192425355

Course Code :- MLA0304

Course Name :- Reinforcement learning

Course Faculty :- Dr. Dumpala Prashanth

CO-1 Assessment - 3

1) Grid World Navigation Using Reinforcement Learning in OpenAI Gymm.

a) Introduction :-

Reinforcement Learning (RL) is a machine learning technique in which an agent learns to make decisions by interacting with an environment. The agent receives rewards or penalties based on its actions and aims to maximize the total reward. In a grid world environment, the objective is to move from the starting position to the goal while avoiding obstacles.

b) Problem Statement :-

Design a grid Environment where:

* The agent starts from a fixed position.
* The goal is to reach the destination.
* Obstacles must be avoided.
* The agent learns the optimal path by maximizing rewards.

c) RL Components

a) State space

The state space consists of all possible positions in the grid.

Reward Function

Action	Reward
Reach Goal	+100
Normal Move	-1
High Energy usage	-5
Collission	-50

Energy Constraints :-

* Battery capacity is limited
* Avoid unnecessary movements
* Robot cannot pass through obstacles
* Reach the goal before the battery is exhausted.

Algorithm :-

* Start the robot at initial position
* Observe the current state and choose the best action
* Move to the next state.
* Update the reward and remaining energy.

Python Program :-

```
energy = 5
while energy > 0;
    print("Energy : ", energy)
    energy == 1
Print("Destination Reached")
```

Sample Output :-

```
Energy = 5
Energy : 4
Energy : 3
Energy : 2
Energy : 1
        Destination reached
```

Advantages	Applications
Minimizes energy consumption	Autonomous Robots
Finds the optimal path	warehouse automation
Improves robot efficiency	Delivery robots
Reduce unnecessary movements	Self-driving vehicles

Conclusion:-

The MDP helps the robot chose the best actions to reach the destination while minimizing energy consumption. It improves efficiency and ensures optimal decision making under battery constraints.

3) Reinforcement Learning Framework for a Warehouse Robot with Battery Constraints

Introduction

Reinforcement learning (RL) enables a warehouse robot to learn the best strategy for completing deliveries while operating within a limited battery capacity. The objective is to maximize deliveries without exceeding the 30-minute battery limit.

Problem Statement

Design an RL framework for a warehouse robot that completes the maximum number of deliveries while ensuring the battery does not exceed 30 minutes.

RL Components

State Space

Robot location

Battery level

Delivery status (Pending / completed)
Action Space
Move forward
Move backward
Pick Package
Deliver Package
Return to charging station
Reward Function

Action	Reward
Successful delivery	+20
Normal Movement	-1
low battery	-10
Battery Exhausted	-50

Constraints :-
* Battery capacity is 30 minutes
* Avoid unnecessary movement
* complete maximum deliveries

4) Algorithm
* Intialize the robot with a 30-minute battery
* Selects the next delivery task
* Move to the destination.
* Deliver the package and receive a award.
* Reduce battery after each action.
* Return to the battery station if the charging is low

5) Python Program:-

```
battery = 30
deliveries = 0
while battery > 5;
      deliveries += 1
```

```
        battery -= 5
Print ("Deliveries : ", deliveries)
Print ("Battery left : ", battery)
```

Sample Output :-
Deliveries : 5
Battery left : 5

Advantages	Applications
Maximizes the number of deliveries.	warehouse automation
Efficient battery usage	logistics Management
Reduces unnecessary movements	Delivery robots
Improves warehouse productivity	Industrial automation

Conclusion:-
The RL framework enables the warehouse robot to maximize deliveries while efficiently managing battery usage. By using rewards and constraints the robot learns an optimal policy that improves productivity and prevents battery exhaustion.

4) Experimental Comparison of Random Action Selection and ε- Greedy Action Selection in a Multi-Armed Bandit Problem

Introduction :-
The Multi Armed Bandit (MAB) problem is a reinforcement learning technique used to maximize rewards by selecting the best action. The question compares Random Action Selection and ε - Greedy Action selection under a fixed time constraints.

Problem statement :-
Compare the performace of Random Action selection and ε - Greedy Action selection by recording cumulative

rewards and identifying better strategy

3. Comparison

Random Action selection	ϵ - Greedy Action selection
Choses actions randomly	Chooses the best action most of time
No learning	learns from previous rewards
High exploration	Balances exploration and explotation

4. Algorithm

1. Intialize the available actions.
2. Select actions randomly or using the ϵ-greedy policy.
3. Receive rewards after each action
4. Update the estimated rewards.
5. Calculate the cumulative reward.
6. Compare both strategies.

5. Python Program

```
import random
random_reward = 0
egreedy_reward = 0
for i in range(10):
    random_reward +=
random.randint(0,1)
    egreedy_reward += 1
Print("random reward :",
random_reward)
Print("Epsilon - Greedy Reward :",
egreedy_reward)
```

Sample Output :-
Random Reward : 6
Epsilon - greedy Reward : 10

Advantages :-
* Random selection :- Simple and easy to implement
* ϵ - greedy : learns the best actions and achieves higher rewards.
* Balances exploration and exploitation effectively

Conclusion :-
The ϵ - greedy Action selection strategy performs better than random actions selection because it learns from previous rewards and choses better actions over time. As a result, it achieves a higher cumulative reward and is more suitable for Reinforcement learning problems.

5) Reinforcement Learning for Traffic Signal Control with Emergency Vehicle Priority

Introduction :-
Reinforcement learning (RL) enables a traffic signal system to optimize traffic flow by learning the best signal timings. The system also gives the immediate priority to emergency vehicles such as ambulances and fire trucks.

Problem Statement :-

Design an RL agent that controls traffic signals while ensuring emergency vehicles receive immediate priority without causing unnecessary traffic congestion

RL Components :-

State space

* Traffic Density
* Signal status (Red / green)
* Emergency Vehicle Presence

Action Space

Green for North - south
Green for east - west
Extend green signal
Change signal.

Reward Function

Action	Reward
Smooth Traffic flow	+10
Emergency vehicle Passes	+20
Traffic Congestion	-5
long waiting Time	-10

Safety Constraints

Emergency vehicles receive priority
Avoid traffic congestion
Prevents signal conflicts

Exploration, Exploitation and Policy learning

Exploration :- Tests different signal timings during normal traffic

Exploitation :- Selects the best timing learned from previous experience

Policy learning :- Updates the traffic signal policy based on rewards while always priortizing emergency vehicles

Python Program:-

```
emergency = True
if emergency:
    print("Green signal for Emergency Vehicle")
else:
    print ("Normal Traffic Signal")
```

Sample Output :-

Green signal for emergency vehicle.

Advantages	Applications
Reduces traffic congestion	Smart cities
Gives priority to emergency vehicles	Traffic management systems
Improves road safety	Emergency response steps.
Optimize traffic signal timing	Intelligent transportation system

Conclusion:-

The RL based traffic signal controller improves traffic flow while ensuring emergency vehicles receive immediate priority. It uses rewards, policy learning and safety constraints to make efficient and safe traffic control decisions